

AI-Assisted Enterprise Code Enhancement Report

Vessel Excel Import — DocumentService

Source

Namespace: Synergy.Rest.Hour.Services.Services

Class: DocumentService

Methods: ImportVesselDataAsync, ImportVesselDataToTable

Stack

.NET · ClosedXML · Entity Framework Core · AutoMapper · Serilog

Tool

Cursor AI / GitHub Copilot assisted refactoring documentation

Date: May 18, 2026

Objective

Demonstrate production-grade enhancement of an enterprise vessel bulk-upload service: structured logging, centralized validation, guard clauses, SOLID alignment, separation of mapping/validation/persistence, and reduced cognitive complexity for large Excel processing.

1. Existing Problems in the Original Code

- `ImportVesselDataAsync` catches all `Exception` types generically; logs via `string-format` template without exception object (loses stack trace in structured sinks).
- User-facing failure message is identical for format errors, DB failures, and corrupt files — hard to support in production.
- `ImportVesselDataToTable` is a single ~400-line method mixing workbook parsing, duplicate detection, validation, mapping, JSON escalation build, repository calls, and response assembly.
- Column index resolution repeats `headersRow.CellsUsed().First(...)` for every column instead of reusing `columnMapping` dictionary.
- Duplicate IMO detection runs twice (`duplicateRows` enumeration + `duplicateVesselImos`) with confusing `duplicatecount` decrement logic.
- `foreach` loop reuses one `vessel` variable; validation blocks (email, IMO, escalation, mandatory columns, company) repeat nearly identical `VesselRequestDto` construction.
- Escalation email validation iterates `escalationLists` twice per row (validate pass + persist pass).
- Nested conditions for existing vessel (`IsOnceApproved`, mapper update, `ExecuteDeleteAsync`) increase cognitive complexity.
- No guard on `fileModel.FileContents` null/empty before `MemoryStream` construction.
- Performance: per-row `FirstOrDefaultAsync` against database; `vesselList.Select.Contains` for IMO deduplication within batch.
- Typos in API surface (`DocumentService`, `UploadFailedReson`, `isAnyInvlaidExist`) reduce maintainability.

2. Original Code – ImportVesselDataAsync

```
public async Task<ResponseWithObject<List<VesselRequestDto>>>> ImportVesselDataAsync(
    FileModelDto fileModel)
{
    var failedRequestList = new ResponseWithObject<List<VesselRequestDto>>>();
    try
    {
        failedRequestList = await ImportVesselDataToTable(fileModel);
    }
    catch (Exception ex)
    {
        Log.Error("Error in {0}.{1}, error description {2}Details{3}",
            MethodBase.GetCurrentMethod()?.DeclaringType?.FullName,
            MethodBase.GetCurrentMethod()?.Name,
            ex.Message,
            ex.InnerException);
        failedRequestList.Status = ResponseStatus.FAILURE.ToString();
        failedRequestList.Message =
            "One or more data in the excel do not match the expected data format. " +
            "Please correct and update again.";
        return failedRequestList;
    }
    return failedRequestList;
}
```

Issues in this entry point

- No validation that fileModel or FileContents is present before calling ImportVesselDataToTable.
- MethodBase.GetCurrentMethod() is fragile under async/optimization; prefer nameof(DocumentService).
- failedRequestList may be partially initialized if exception occurs before assignment.

3. Original Code — ImportVesselDataToTable (structure)

```
private async Task<ResponseWithObject<List<VesselRequestDto>>> ImportVesselDataToTable(
    FileModelDto fileModel)
{
    var serviceResponse = new ResponseWithObject<List<VesselRequestDto>>();
    Stream stream = new MemoryStream(fileModel.FileContents);
    using (XLWorkbook workBook = new XLWorkbook(stream))
    {
        IXLWorksheet worksheet = workBook.Worksheet(1);
        // Header validation + requiredColumns → columnMapping
        // Duplicate IMO: GroupBy on worksheet rows (computed twice)
        // Load companyIds, vesselTypeList, contractTypeList, flagList (good)
        // foreach (var row in worksheet.RowsUsed().Skip(1).Except(duplicateRows))
        // { map cells → validate email → map FKs → validate escalation
        //   → mandatory columns → IMO → company → DB match/update/create
        //   → pending tasks + escalation emails }
        // SaveAllAsync / UpdateRangeAsync / bulk pending / escalation save
        // Build partial-failure or success Response
    }
}
```

Header & duplicate handling excerpt

```
var duplicateRows = worksheet.RowsUsed().Skip(1)
    .Where(row => !string.IsNullOrEmpty(row.Cell(rownum).Value.ToString()))
    .GroupBy(row => row.Cell(rownum).Value.ToString())
    .Where(group => group.Count() > 1)
    .SelectMany(group => group);

var duplicateVesselImos = worksheet.RowsUsed().Skip(1)
    .Where(...)
    .GroupBy(...)
    .Where(group => group.Count() > 1)
    .Select(group => group.Key)
    .ToList();
```

4. Original Code — Row loop validation (repetitive)

```
foreach (var row in worksheet.RowsUsed().Skip(1).Except(duplicateRows))
{
    vessel = new Vessel();
    vessel.Id = Guid.NewGuid();
    vessel.CompanyId = row.Cell(company_ID).Value.ToString();
    vessel.VesselName = row.Cell(ship_Name).Value.ToString();
    vessel.VesselImo = row.Cell(ship_IMO_No).Value.ToString();
    vessel.VesselEmail = row.Cell(ship_Email).Value?.ToString().Trim() ?? string.Empty;

    isValidMail = EmailExtensions.IsValidEmail(vessel.VesselEmail);
    if (!isValidMail)
    {
        failedRequestList.Add(new VesselRequestDto {
            CompanyId = vessel.CompanyId,
            VesselName = vessel.VesselName,
            BatchId = vessel.BatchId,
            UploadFailedReason = $"Invalid ship email: {vessel.VesselEmail}"
        });
        continue;
    }
    // ... similar blocks for escalation emails, empty columns,
    //     IMO validation, company ID, duplicate approved IMO ...
    validImo = ValidatorExtension.IsValid7DigitInteger(
        vessel.VesselImo ?? string.Empty, out int result);
    if (!validImo) { failedRequestList.Add(...); continue; }
}
```

Why this hurts maintainability

- Six+ separate validation branches each construct VesselRequestDto with the same three fields.
- Business rules (email, IMO, company, escalation) are not unit-testable in isolation.
- Order of validation is implicit; changing one rule risks inconsistent error messages.

5. AI Prompt Used

Enhance this enterprise C# service implementation (DocumentService vessel Excel import) using modern .NET clean architecture principles.

Focus areas:

- structured exception logging (Serilog)
- reusable validation methods
- guard clauses and null safety
- SOLID principles (especially Single Responsibility)
- separation of concerns (parse / validate / map / persist)
- helper method extraction
- maintainability and reduced cognitive complexity
- performance optimization for large Excel processing
- production-ready coding standards

Preserve existing API contracts and response DTO shapes.

6. AI-Enhanced — ImportVesselDataAsync

```
public async Task<ResponseWithObject<List<VesselRequestDto>>> ImportVesselDataAsync(
    FileModelDto fileModel)
{
    if (fileModel?.FileContents == null || fileModel.FileContents.Length == 0)
        return CreateFailureResponse(
            "Uploaded file is empty or invalid.");

    try
    {
        return await ImportVesselDataToTable(fileModel);
    }
    catch (FormatException ex)
    {
        Log.Error(ex,
            "Validation error in {Service}.{Method}",
            nameof(DocumentService),
            nameof(ImportVesselDataAsync));
        return CreateFailureResponse(
            "Excel contains invalid data format. Please correct and upload again.");
    }
    catch (Exception ex)
    {
        Log.Error(ex,
            "Unexpected error in {Service}.{Method}",
            nameof(DocumentService),
            nameof(ImportVesselDataAsync));
        return CreateFailureResponse(
            "Unexpected error occurred while importing vessel data.");
    }
}

private static ResponseWithObject<List<VesselRequestDto>> CreateFailureResponse(
    string message) => new()
{
    Status = ResponseStatus.FAILURE.ToString(),
    Message = message,
    Response = new List<VesselRequestDto>()
};
```

7. AI-Enhanced — Extracted validation & failure DTO

```
private static VesselRequestDto CreateFailedRequest(
    Vessel vessel, string reason) => new()
{
    CompanyId = vessel.CompanyId,
    VesselName = vessel.VesselName,
    BatchId = vessel.BatchId,
    UploadFailedReason = reason
};

private bool TryValidateVesselRow(
    Vessel vessel,
    string escalationRequirement,
    IReadOnlyList<string> invalidEscalationEmails,
    IReadOnlyList<CompanyRef> companyIds,
    out string errorMessage)
{
    errorMessage = string.Empty;

    if (!EmailExtensions.IsValidEmail(vessel.VesselEmail))
    {
        errorMessage = $"Invalid ship email: {vessel.VesselEmail}";
        return false;
    }
    if (invalidEscalationEmails.Count > 0)
    {
        errorMessage = $"Invalid Escalation email: " +
            string.Join(", ", invalidEscalationEmails);
        return false;
    }
    var emptyColumns = CheckAndReturnEmptyColumns(
        vessel.CompanyId ?? string.Empty,
        vessel.VesselName ?? string.Empty,
        vessel.VesselImo ?? string.Empty,
        vessel.VesselEmail,
        escalationRequirement);
    if (emptyColumns.Count > 0)
    {
        errorMessage = "The following mandatory columns are empty or null: " +
            string.Join(", ", emptyColumns);
        return false;
    }
    if (!ValidatorExtension.IsValid7DigitInteger(
        vessel.VesselImo ?? string.Empty, out _))
    {
        errorMessage = $"Invalid ship IMO No: {vessel.VesselImo}";
        return false;
    }
    if (!companyIds.Any(c => c.CompanyId == vessel.CompanyId))
    {
        errorMessage = "Company ID not matching.";
        return false;
    }
    return true;
}
```

Escalation validation extracted per row

```
private static (bool HasEscalation, List<string> InvalidEmails) ValidateEscalationEmails(
    IXLRow row, IReadOnlyList<int> escalationColumns)
{
    var invalid = new List<string>();
    var hasAny = false;
    foreach (var col in escalationColumns)
    {
        var email = row.Cell(col).Value?.ToString().Trim() ?? string.Empty;
        if (string.IsNullOrEmpty(email)) continue;
        hasAny = true;
        if (!EmailExtensions.IsValidEmail(email))
            invalid.Add(email);
    }
    return (hasAny, invalid);
}
```

8. AI-Enhanced – Simplified row loop

```
var processedImosInBatch = new HashSet<string>(StringComparer.OrdinalIgnoreCase);

foreach (var row in dataRows)
{
    var vessel = MapVesselFromRow(row, columnIndexes, batchId, _applicationUser.Id);

    var (hasEscalation, invalidEscalation) =
        ValidateEscalationEmails(row, escalationColumns);

    if (!TryValidateVesselRow(vessel, hasEscalation ? "Exist" : string.Empty,
        invalidEscalation, companyIds, out var validationError))
    {
        failedRequestList.Add(CreateFailedRequest(vessel, validationError));
        continue;
    }

    if (!processedImosInBatch.Add(vessel.VesselImo!.Trim()))
        continue; // duplicate within same upload batch

    vessel.CompanyUniqueRefId = ResolveCompanyId(companyIds, vessel.CompanyId);
    vessel.PreviousApprovedCompanyUniqueRefId = vessel.CompanyUniqueRefId;
    ApplyLookupIds(vessel, vesselTypeList, contractTypeList, flagList);

    var existing = await FindExistingVesselAsync(vessel.VesselImo);
    if (existing?.IsOnceApproved == true)
    {
        failedRequestList.Add(CreateFailedRequest(vessel,
            $"Ship IMO No {vessel.VesselImo} is already in use."));
        continue;
    }

    vessel.EscalationEmailDocument = BuildEscalationJson(row, escalationColumns);
    if (existing != null)
        await ApplyUpdateAsync(existing, vessel, vesselUpdateList);
    else
        AddNewVessel(vessel, vesselList);

    AddPendingAndEscalationEmails(row, vessel, pendingList, escalationMail, escalationColumns);
}
```

Structural refactor recommendations

- Split `ImportVesselDataToTable` into: `ReadWorkbookContext`, `ProcessRowsAsync`, `PersistImportResultsAsync`.
- Pre-compute duplicate IMOs once; emit one failure row per duplicate key.
- Batch DB read: load existing vessels by IMO set from Excel before loop.

9. Expert-Level Improvements Introduced

Area	Improvement
Guard clauses	Reject null/empty FileContents before MemoryStream and workbook parsing.
Structured logging	Log.Error(ex, ...) with Service and Method tokens for Serilog enrichment.
Response factory	CreateFailureResponse eliminates duplicated Status/Message/Response initialization.
Validation extraction	TryValidateVesselRow + ValidateEscalationEmails – testable, single exit per rule.
Failure DTO factory	CreateFailedRequest removes six copy-pasted object initializers.
Separation of concerns	MapVesselFromRow, BuildEscalationJson, ApplyUpdateAsync isolate responsibilities.
Performance	HashSet for in-batch IMO; optional bulk IMO query instead of per-row FirstOrDefaultAsync.
SOLID (SRP)	Import method orchestrates; helpers own one reason to change.
Exception specificity	FormatException vs general Exception yields accurate user messaging.

10. Productivity Observation

- AI-assisted development significantly reduced refactoring effort for repetitive validation, response handling, and Serilog patterns across ImportVesselDataAsync and the row loop.
- Cursor AI / GitHub Copilot accelerated clean-code transformation by suggesting reusable helper methods, structured logging, and maintainable enterprise refactoring in minutes instead of hours for a method that would otherwise require careful manual decomposition of ~400 lines.

Reference

Original: Synergy.Rest.Hour.Services.Services.DocumentService
Dependencies: ClosedXML, EF Core, AutoMapper, Serilog, Newtonsoft.Json